

```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
```

```
# Set random seeds for reproducibility
torch.manual_seed(42)
```

```
↗ <torch._C.Generator at 0x7e0f94dc35b0>
```

```
# ● Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

```
↗ Using device: cuda
```

```
df = pd.read_csv('fashion-mnist_train.csv')
df.head()
```

```
↗
```

|   | label | pixel1 | pixel2 | pixel3 | pixel4 | pixel5 | pixel6 | pixel7 | pixel8 | pixel9 | ... | pixel775 | pixel776 | pixel777 | pixel778 | pixel779 | p |
|---|-------|--------|--------|--------|--------|--------|--------|--------|--------|--------|-----|----------|----------|----------|----------|----------|---|
| 0 | 2     | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | ... | 0        | 0        | 0        | 0        | 0        |   |
| 1 | 9     | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | ... | 0        | 0        | 0        | 0        | 0        |   |
| 2 | 6     | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 5      | 0      | ... | 0        | 0        | 0        | 30       | 43       |   |
| 3 | 0     | 0      | 0      | 0      | 1      | 2      | 0      | 0      | 0      | 0      | ... | 3        | 0        | 0        | 0        | 0        |   |
| 4 | 3     | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | ... | 0        | 0        | 0        | 0        | 0        |   |

5 rows × 785 columns

```
df.shape
```

```
↗ (60000, 785)
```

```
# Create a 4x4 grid of images
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
fig.suptitle("First 16 Images", fontsize=16)
```

```
# Plot the first 16 images from the dataset
for i, ax in enumerate(axes.flat):
    img = df.iloc[i, 1:].values.reshape(28, 28) # Reshape to 28x28
    ax.imshow(img) # Display in grayscale
    ax.axis('off') # Remove axis for a cleaner look
    ax.set_title(f"Label: {df.iloc[i, 0]}") # Show the label
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96]) # Adjust layout to fit the title
plt.show()
```



## First 16 Images



```
# train test split
```

```
X = df.iloc[:, 1:].values  
y = df.iloc[:, 0].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train = X_train/255.0  
X_test = X_test/255.0
```

```
class CustomDataset(Dataset):
```

```
    def __init__(self, features, labels):
```

```
        # Convert to PyTorch tensors
```

```
        self.features = torch.tensor(features, dtype=torch.float32).reshape(-1,1,28,28)  
        self.labels = torch.tensor(labels, dtype=torch.long)
```

```
    def __len__(self):
```

```
        return len(self.features)
```

```
def __getitem__(self, index):
    return self.features[index], self.labels[index]
```

```
train_dataset = CustomDataset(X_train, y_train)
```

```
test_dataset = CustomDataset(X_test, y_test)
```

```
# ● pin_memory used for train fast
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, pin_memory=True)
```

```
len(train_loader)
```

```
↔ 1500
```

```
class MyNN(nn.Module):
    def __init__(self, input_features):
        super().__init__()
        # feature Extraction
        self.model = nn.Sequential(
            nn.Conv2d(input_features, 32, kernel_size=3, padding='same'),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(32, 64, kernel_size=3, padding='same'),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # Feature Classification
        self.classifier= nn.Sequential(
            nn.Flatten(),
            # find tensor size which is gonna for flatten
            # input -> convolution(32 filter, size=3*3) -> MaxPooling, size(2*2) -> convolution2layer(64 filters, size(3*3)) -> maxPooling2 (size(1,2,2,2)) -> (32,28,28) -> (32,14,14) -> (64,14,14) -> (64,7,7)
            nn.Linear(64*7*7, 128),
            nn.Dropout(p=0.4),
            nn.ReLU(),
            nn.Dropout(p=0.4),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        X= self.model(x)
        X= self.classifier(X)
        return X
```

```
learning_rate = 0.1
epochs = 100
```

```
# instantiate the model
model = MyNN(1)
model = model.to(device)
# loss function
criterion = nn.CrossEntropyLoss()
# optimizer
optimizer = optim.SGD(model.parameters(), lr=learning_rate, weight_decay=1e-4 )
```

```
# training loop
```

```
for epoch in range(epochs):
```

```
    total_epoch_loss = 0
```

```
    for batch_features, batch_labels in train_loader:
```

```
        # ● move data to gpu
        batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)
```

```

# forward pass
outputs = model(batch_features)

# calculate loss
loss = criterion(outputs, batch_labels)

# back pass
optimizer.zero_grad()
loss.backward()

# update grads
optimizer.step()

total_epoch_loss = total_epoch_loss + loss.item()

avg_loss = total_epoch_loss/len(train_loader)
print(f'Epoch: {epoch + 1} , Loss: {avg_loss}')
```

```

↔ Epoch: 43 , Loss: 0.1394271306854207
Epoch: 44 , Loss: 0.13383356608178776
Epoch: 45 , Loss: 0.1324825570284544
Epoch: 46 , Loss: 0.13166040645345736
Epoch: 47 , Loss: 0.1308607600585868
Epoch: 48 , Loss: 0.12954192445605683
Epoch: 49 , Loss: 0.12984606293852752
Epoch: 50 , Loss: 0.12873638029846673
Epoch: 51 , Loss: 0.1278521613473616
Epoch: 52 , Loss: 0.12777134339216475
Epoch: 53 , Loss: 0.12011745175618369
Epoch: 54 , Loss: 0.1246262890966221
Epoch: 55 , Loss: 0.12669147541544712
Epoch: 56 , Loss: 0.11947256458519648
Epoch: 57 , Loss: 0.12252410604680578
Epoch: 58 , Loss: 0.12046081719345723
Epoch: 59 , Loss: 0.11775021589747242
Epoch: 60 , Loss: 0.11621114240012442
Epoch: 61 , Loss: 0.11790137830351402
Epoch: 62 , Loss: 0.1209013796785924
Epoch: 63 , Loss: 0.11556983594714741
Epoch: 64 , Loss: 0.11551602995059027
Epoch: 65 , Loss: 0.11448292561051979
Epoch: 66 , Loss: 0.11447305859733994
Epoch: 67 , Loss: 0.10922846284128415
Epoch: 68 , Loss: 0.114074466937338
Epoch: 69 , Loss: 0.11219523354140498
Epoch: 70 , Loss: 0.10807277393628223
Epoch: 71 , Loss: 0.11037747984482363
Epoch: 72 , Loss: 0.11096230870733659
Epoch: 73 , Loss: 0.11467301291493155
Epoch: 74 , Loss: 0.10951105096567577
Epoch: 75 , Loss: 0.11254930374978964
Epoch: 76 , Loss: 0.10673036707895033
Epoch: 77 , Loss: 0.10492317223419863
Epoch: 78 , Loss: 0.10162908646600166
Epoch: 79 , Loss: 0.10138705732199985
Epoch: 80 , Loss: 0.1065858583059938
Epoch: 81 , Loss: 0.10710911753688318
Epoch: 82 , Loss: 0.10254762729241823
Epoch: 83 , Loss: 0.1064309622043899
Epoch: 84 , Loss: 0.1049523995364434
Epoch: 85 , Loss: 0.10229275975880349
Epoch: 86 , Loss: 0.10323731615875537
Epoch: 87 , Loss: 0.10259687216339808
Epoch: 88 , Loss: 0.0992780900390353
Epoch: 89 , Loss: 0.10130338226911166
Epoch: 90 , Loss: 0.09864793314656709
Epoch: 91 , Loss: 0.09973211605542262
Epoch: 92 , Loss: 0.10374507245464096
Epoch: 93 , Loss: 0.09732181633794486
Epoch: 94 , Loss: 0.10046478798838022
Epoch: 95 , Loss: 0.09992808149105016
Epoch: 96 , Loss: 0.0988437332607573
Epoch: 97 , Loss: 0.10111900116939311
Epoch: 98 , Loss: 0.09593968087178655
Epoch: 99 , Loss: 0.09846772438043262
Epoch: 100 , Loss: 0.0978204097916314
```

```
model.eval()
```

```

↔ MyNN(
  (model): Sequential(
```

```

(0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=same)
(1): ReLU()
(2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=same)
(5): ReLU()
(6): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(classifier): Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=3136, out_features=128, bias=True)
  (2): Dropout(p=0.4, inplace=False)
  (3): ReLU()
  (4): Dropout(p=0.4, inplace=False)
  (5): Linear(in_features=128, out_features=10, bias=True)
)
)

```

# evaluation code for test dataset

```

total = 0
correct = 0

```

with torch.no\_grad():

for batch\_features, batch\_labels in test\_loader:

```

# ● move data to gpu
batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)

outputs = model(batch_features)

_, predicted = torch.max(outputs, 1)

total = total + batch_labels.shape[0]

correct = correct + (predicted == batch_labels).sum().item()

```

print(correct/total)

→ 0.92275

# evaluation code for train dataset

```

total = 0
correct = 0

```

with torch.no\_grad():

for batch\_features, batch\_labels in train\_loader:

```

# ● move data to gpu
batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)

outputs = model(batch_features)

_, predicted = torch.max(outputs, 1)

total = total + batch_labels.shape[0]

correct = correct + (predicted == batch_labels).sum().item()

```

print(correct/total)

→ 0.9908333333333333

Start coding or [generate](#) with AI.